



Laboratório - 2: Implementação de tarefas

Professor: Rogério Pereira Junior

rogerio.pereira@ifsc.edu.br

PARTE III - Threads

Crie um documento de texto para responder as perguntas e inserir as capturas de tela requeridas durante o experimento

Objetivos

- Compreender a criação e o ciclo de vida de Threads em C (Pthreads).
- Analisar o escalonamento, prioridades e estados de processos via Shell.
- Observar problemas de concorrência (Race Conditions).
- Praticar o controle de jobs (Foreground/Background) e sinais de terminação.

Organização

- Temos diversos Experimentos sobre criação de processos e Threads
- Os iniciais são referentes a criação e análise de processos com `fork()` e `execve()`
- Os demais são referentes a criação e análise de Threads
- Será requerido capturas de tela sobre algumas etapas de análise

Experimento 1: Criando processos com `fork()` e analisando com `ps`

Passo 1: Criar o código em C

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid == 0) {
        // Processo filho
        printf("Sou o processo FILHO. PID = %d\n", getpid());
        sleep(30);
    } else {
        // Processo pai
        printf("Sou o processo PAI. PID = %d, filho = %d\n", getpid(), pid);
        sleep(30);
    }

    return 0;
}
```

Passo 2: Compilar o programa

```
1 gcc fork_exemplo.c -o fork_exemplo
```

Passo 3: Executar o programa

```
1 ./fork_exemplo
```

Passo 4: Em outro terminal, execute:

```
ps -f
```

Atividade:

- Tire um print da saída do comando `ps -f`
- Identifique o processo pai e o processo filho
- Explique como você conseguiu diferenciar os dois

Experimento 2: fork() + execve() e análise com ps

Passo 1: Criar o código em C

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Filho executa outro programa
        char *args[] = {"/bin/ls", "-l", NULL};
        execve("/bin/ls", args, NULL);
    } else {
        // Pai permanece ativo
        printf("Processo pai PID = %d\n", getpid());
        sleep(30);
    }

    return 0;
}
```

Passo 2: Compilar

```
1 gcc exec_exemplo.c -o exec_exemplo
```

Passo 3: Executar

```
1 ./exec_exemplo
```

Passo 4: Monitorar processos

```
1 ps -f
```

Atividade:

- Tire um print da saída do `ps -f`
- Identifique o processo que executou o comando `ls`
- Explique o que aconteceu com o código original do filho após o `execve()`
- Compare com o comportamento da Parte 1

Experimento 3: Criando múltiplos processos e visualizando com `ps-tree + grep`

Passo 1: Criar o código em C

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int i;

    for (i = 0; i < 3; i++) {
        fork();
    }

    printf("Processo PID = %d\n", getpid());
    sleep(30);

    return 0;
}
```

Observação: Cada chamada de `fork()` duplica os processos existentes.

Passo 2: Compilar

```
1 gcc multi_fork.c -o multi_fork
```

Passo 3: Executar

```
1 ./multi_fork
```

Passo 4: Visualizar árvore de processos

```
1 pstree | grep multi_fork
```

OU (mais detalhado):

```
1 pstree -p | grep multi_fork
```

Atividade:

- Tire um print da saída do `pstree`
- Conte quantos processos foram criados
- Explique por que esse número de processos foi gerado
- Desenhe a árvore de processos manualmente

Experimento 4: Quantidade de Processos e Saídas na Tela

O objetivo aqui é compreender como múltiplas chamadas de `fork()` impactam a quantidade de processos e saídas exibidas.

Código 1

```
#include <stdio.h>
#include <unistd.h>
int main() {
    fork();
    fork();
    printf("Oi\n");
    return 0;
}
```

Atividades

- Sem executar o código, responda:
 - Quantos processos serão criados ao final da execução?
 - Quantas vezes a mensagem "Oi" será exibida?
- Desenhe a árvore de processos gerada (pai e filhos).
- Execute o código e verifique sua resposta:

```
1 gcc exemplo1.c -o exemplo1
2 ./exemplo1
```

- Capture a tela da execução;
- Compare o resultado com sua previsão e explique.

Código 2

```
3 #include <stdio.h>
4 #include <unistd.h>
5 int main() {
6     fork();
7     fork();
8     fork();
9     printf("Oi\n");
10    return 0;
11 }
```

Atividades

- Sem executar:
 - Quantos processos serão criados?
 - Quantas linhas "Oi" aparecerão?
- Desenhe a árvore de processos;

- Execute e valide:

```
gcc exemplo2.c -o exemplo2  
./exemplo2
```

- Capture a tela;
- Explique o crescimento exponencial dos processos.

Experimento 5: Sincronização com `wait()`

Demonstrar a importância da chamada de sistema `wait()` para a sincronização entre processos pai e filho, evitando o surgimento de processos zumbis e garantindo a ordem de execução.

Código Base: O Filho Preguiçoso

Neste programa, o filho levará 5 segundos para "trabalhar", enquanto o pai aguardará pacientemente antes de encerrar.

```
14 #include <stdio.h>
15 #include <stdlib.h>
16 #include <unistd.h>
17 #include <sys/wait.h>
18
19 int main() {
20     pid_t pid;
21     int status;
22
23     pid = fork();
24
25     if (pid < 0) {
26         perror("Erro no fork");
27         exit(1);
28     }
29
30     if (pid == 0) {
31         // Bloco do FILHO
32         printf("[FILHO]: Iniciando tarefa pesada (5 segundos)...\n");
33         sleep(5);
34         printf("[FILHO]: Tarefa concluída. Vou encerrar agora com status 42.\n");
35         exit(42);
36     } else {
37         // Bloco do PAI
38         printf("[PAI]: Meu filho (PID %d) está trabalhando. Vou aguardar...\n", pid);
39
40         // O pai para aqui ate o filho dar exit()
41         wait(&status);
42
43         if (WIFEXITED(status)) {
44             printf("[PAI]: Meu filho terminou normalmente com status: %d\n", WEXITSTATUS(status));
45         }
46         printf("[PAI]: Agora que meu filho encerrou, eu também vou. Tchau!\n");
47     }
48
49     return 0;
50 }
```

Atividade

1. Compile o código: `gcc lab_wait.c -o lab_wait`
2. Execute o programa e observe o tempo de espera no terminal.
3. **Desafio:** Comente a linha 28 (`wait(&status);`), compile e rode novamente.

1 Questões para o Relatório

1. No experimento com o `wait()` ativo, quem imprime a última mensagem no terminal? Por quê?

2. Quando você comentou o `wait()`, o que aconteceu com a ordem das mensagens? O pai esperou o filho?
3. O que são as macros `WIFEXITED` e `WEXITSTATUS`? Para que servem?
4. Explique por que o `wait()` é fundamental para evitar que o sistema fique cheio de processos "zumbis".

Experimento 6: A Primeira Thread (Simples)

Este código apenas cria uma thread que imprime mensagens. O objetivo é usar o `ps` e `lsuf`.

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

void* print_hello(void* arg) {
    while(1) {
        printf("Olá da Thread! (PID: %d)\n", getpid());
        sleep(5);
    }
    return NULL;
}

int main() {
    pthread_t thread_id;
    printf("Thread Principal iniciada. PID: %d\n", getpid());
    pthread_create(&thread_id, NULL, print_hello, NULL);
    pthread_join(thread_id, NULL);
    return 0;
}
```

Tarefa Linux:

1. Execute o programa.

```
gcc threads.c -o exp01 -lpthread
```

2. Use `ps -eL | grep exp01` para ver a thread.
3. Use `lsuf -p [PID]` para ver quais arquivos o processo abriu.
4. Faça Capturas de tela para os comandos dos itens 2 e 3

Experimento 7: O Contador de Threads

O código abaixo cria 4 threads, cada uma com um tempo de vida diferente. Note o uso da biblioteca `unistd.h` para a função `sleep()`.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

void* tarefa_thread(void* arg) {
    int id = *((int*)arg);
    int tempo = id * 30; // 30s, 60s, 90s, 120s

    printf("Thread %d iniciada. Vou rodar por %d segundos.\n", id, tempo);
    sleep(tempo);
    printf("Thread %d finalizada após %d segundos.\n", id, tempo);

    pthread_exit(NULL);
}

int main() {
    pthread_t threads[4];
    int ids[4];

    printf("Processo Principal (PID: %d) iniciando...\n", getpid());

    for (int i = 0; i < 4; i++) {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, tarefa_thread, &ids[i]);
    }

    for (int i = 0; i < 4; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Todas as threads terminaram. Encerrando processo principal.\n");
    return 0;
}
```

Compilação: Utilize a flag `-lpthread`:

```
gcc threads_vida.c -o threads_vida -lpthread
```

Usando o Htop

Para visualizar as threads corretamente, siga estes passos no `htop`:

1. Abra o `htop` em um terminal separado.
2. Pressione **F2** (Setup) → **Display options**.
3. Marque a opção **"Three view"** (Visualização em árvore) e certifique-se de que **"Hide userland process threads"** esteja **desmarcado**. Pressione F10 para salvar.
4. Pressione **F4** (Filter) e digite `threads_vida`. Agora você verá apenas o seu processo e suas threads filhas.

Execução e Observação

- Execute o programa
- Alterne para o terminal com o `htop`.

- Observe que inicialmente aparecem 5 linhas (1 processo pai e 4 threads).
- Faça uma captura de tela

Responda

1. Descreva o que acontece no `htop` a cada intervalo de 30 segundos. As linhas desaparecem ou mudam de cor?
2. No `htop`, as threads possuem o mesmo PID (Process ID) do processo pai?
3. Por que o processo principal (pai) só termina após a última thread (de 120s) finalizar?
4. Se você fechar o programa com `Ctrl+C`, o que acontece com as threads que ainda estavam rodando?

Experimento 8: Concorrência e Race Condition

Duas threads tentam incrementar a mesma variável global 1 milhão de vezes.

```
#include <pthread.h>
#include <stdio.h>

long contador = 0;

void* incrementar(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        contador++; // Regiao Critica sem exclusao mutua
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, incrementar, NULL);
    pthread_create(&t2, NULL, incrementar, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Valor final do contador: %ld (Esperado: 2000000)\n", contador);
    return 0;
}
```

Tarefa Linux:

1. Execute o código várias vezes.
2. Faça algumas capturas de tela dos resultados
3. O resultado muda? Por que?
4. Use o comando `time ./programa` para medir o tempo de execução.

Experimento 9: Monitoramento de Longa Duração

Três threads com comportamentos distintos para análise no top e htop.

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <math.h>

void* cpu_intensa(void* arg) {
    while(1) { sqrt(rand()); } // Consome 100% de um nucleo
}

void* entrada_saida(void* arg) {
    while(1) { printf("Log de Operacao...\n"); sleep(1); }
}

int main() {
    pthread_t t1, t2, t3;
    pthread_create(&t1, NULL, cpu_intensa, NULL);
    pthread_create(&t2, NULL, cpu_intensa, NULL);
    pthread_create(&t3, NULL, entrada_saida, NULL);
    pthread_join(t1, NULL);
    return 0;
}
```

Tarefa Linux:

1. Execute e abra o htop.
2. Pressione F2 -> Display Options -> Marque "Tree View".
3. Use F4 para filtrar os processos
4. Use renice para mudar a prioridade desse processo enquanto ele roda.
5. Faça captura de telas dos itens anteriores

Experimento 10: Simulação de Servidor Web

Simula um Dispatcher distribuindo "tarefas" para Workers.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void* worker(void* id) {
    int num = *(int*)id;
    while(1) {
        printf("[Worker %d] Aguardando requisicao...\n", num);
        sleep(rand() % 5 + 1);
        printf("[Worker %d] Processando...\n", num);
    }
}

int main() {
    pthread_t threads[5];
    int ids[5];
    for(int i=0; i<5; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }
    printf("Servidor rodando com 5 Workers. PID: %d\n", getpid());
    while(1) { sleep(10); } // Mantem o main vivo
    return 0;
}
```

Tarefa Linux:

1. Coloque o servidor em background (CTRL+Z seguido de bg).
2. Use kill -9 [PID] para "derrubar" o servidor.

Desafio Final (Opcional): Análise Teórica de Modelos de Threads

Considere o trecho de código abaixo para responder às questões de análise de modelos de mapeamento de threads (**N:1** - Muitos para Um e **1:1** - Um para Um).

```
171 int y = 0;
172
173 void threadBody() {
174     int x = 0;
175     sleep(10);
176     printf("x: %d, y: %d\n", ++x, ++y);
177     thread_exit();
178 }
179
180 main() {
181     thread_create(&tA, threadBody, ...);
182     thread_create(&tB, threadBody, ...);
183     sleep(1);
184     thread_join(&tA);
185     thread_join(&tB);
186     sleep(1);
187     thread_create(&tC, threadBody, ...);
188     thread_join(&tC);
189 }
```

Informações Importantes:

- A operação `sleep()` é uma chamada de sistema (syscall) bloqueante.
- No modelo **N:1**, se uma thread faz uma syscall bloqueante, o processo inteiro (e todas as suas threads) é bloqueado pelo Kernel.
- No modelo **1:1** (padrão Linux moderno), o Kernel gerencia cada thread individualmente.

Pede-se:

A) Diagramas de Execução

Desenhe o diagrama de tempo para os dois modelos (N:1 e 1:1), mostrando a execução das threads ao longo do tempo.

B) Durações Aproximadas

Informe o tempo total aproximado de execução do programa completo nos dois cenários:

1. Modelo N:1
2. Modelo 1:1

C) Saída do Programa

Indique exatamente o que aparecerá na tela (o conteúdo dos `printf`) ao final da execução. A saída difere entre os modelos? Justifique considerando o escopo das variáveis `x` e `y`.